# Lecture 2 — Implementation & Simulation

> Regional Level · Prerequisites: Year Curriculum Weeks 2 (STL), 16 (grids, BFS), 35 (debugging) · Estimated time: 2 hours

## 1. Overview

Almost every regional contains one to three problems whose algorithm is obvious: "simulate the robot", "evaluate the formula", "apply the voting rules". Strong teams still lose large amounts of time on them, because the danger is not the idea but the details. A 40-minute implementation with two wrong submissions costs 80 minutes of penalty, more than an entire medium problem solved an hour late.

This lecture is about engineering under pressure:

- how to recognise an implementation problem and plan it on paper,
- techniques that shrink code (direction arrays, symmetry by rotation),
- robust parsing, including recursive descent for nested syntax,
- simulating astronomically many steps using cycle detection,
- event-driven simulation with well-defined tie-breaking.

## 2. Recognising and planning

Signals of an implementation problem: a long statement full of rules, small constraints (often the brute force *is* the solution), words like *simulate*, *process*, *commands*, *game*, and an intricate output format.

Before typing, spend about five minutes writing:

1. **The state.** Which variables completely describe the situation? For a robot it might be (row, column, direction, number of collected items).
2. **The transitions.** One function per rule in the statement. Name them.
3. **The representation.** Prefer arrays to maps and integers to strings. Convert input into a convenient internal form immediately (e.g. letters to 0-based indices).
4. **Corner cases** listed explicitly from the statement: every "if", "unless", "except" deserves a line.
5. **The output format** copied exactly.

This plan doubles as a test list. It also makes it possible for a teammate to take over if you get stuck.

# 3. Techniques that shrink code

## 3.1 Direction arrays

Four copy-pasted `if` blocks for up/down/left/right are a classic source of bugs: you fix a mistake in three of them and forget the fourth. Use arrays:

```
const int dr[4] = {-1, 0, 1, 0};   // N, E, S, W
const int dc[4] = {0, 1, 0, -1};
// turn right: d = (d + 1) % 4; turn left: d = (d + 3) % 4; reverse: d ^ 2
```

With directions ordered clockwise, turning and reversing become arithmetic. The same idea covers 8-neighbourhoods, knight moves, and hexagonal grids (six offsets, with parity-dependent arrays in offset coordinates).

## 3.2 Symmetry by rotation

When a problem asks for the same operation in four directions, implement only one and transform the input. In the 2048 game (Kattis `2048`), you are given a 4×4 board and a direction; all tiles slide in that direction and equal neighbours merge once. Implement only "slide left" on a single row. For the direction "up", rotate the board 90° counter-clockwise once (the top row becomes the left column), slide left, then rotate three more times to get back. In general, direction index *d* (0 = left, 1 = up, 2 = right, 3 = down) means *d* rotations before and *(4 − d) mod 4* after.

Within the row, structure the work in three phases rather than one tricky loop: **compress** (drop zeros), **merge** (scan, merging equal adjacent pairs, skipping the second element of a merge so each tile merges once), **pad** with zeros. The full program is in Section 6.

The same principle appears elsewhere: reverse a string to turn "suffix" logic into "prefix" logic, transpose a matrix to turn column operations into row operations, or reflect coordinates so that only one quadrant needs handling.

# 4. Parsing

Choose the tool by the shape of the input:

| Input shape | Tool |
|---|---|
| Whitespace-separated tokens | `cin >> x` |
| Lines containing spaces | `getline(cin, line)` |
| Tokens inside a line | `stringstream ss(line); while (ss >> w) ...` |
| Fixed punctuation (`12:34`, `3/4`) | `scanf("%d:%d", &h, &m)` or read a char separator |

| Input shape | Tool |
|---|---|
| Nested syntax (expressions, formulas, trees) | recursive descent |

A frequent bug: after `cin >> n`, the newline is still in the buffer, so the next `getline` returns an empty string. Call `cin.ignore()` or a throwaway `getline` first.

### 4.1 Example: Codeforces 1B — Spreadsheets

Cells are written either as `BC23` (column letters, row number) or `R23C55`. Convert each to the other form. Two traps:

- **Column letters are bijective base 26**: there is no zero digit, `Z` = 26, `AA` = 27. To convert a number to letters, decrement before each `% 26`.
- **Detecting the format**: `R23C55` starts with `R`, has a digit right after it, and contains a later `C`. A string like `RC23` is in letters-then-digits form (column `RC`, row 23), so checking only "starts with R and contains C" is wrong.

### 4.2 Recursive descent

For nested syntax, write the grammar first, then one function per rule:

```
expr   := term   (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := '-' factor | '(' expr ')' | number
```

Precedence falls out of the nesting: `*` binds tighter than `+` because terms are built before expressions combine them. Each function consumes characters from a shared position. Left-associativity comes from the `while` loop (not from recursion like `expr := term '+' expr`, which would make `8 - 3 - 2` evaluate as `8 - (3 - 2)`).

The same skeleton parses chemical formulas (`Mg(OH)2`), tree descriptions (`(a(b)(c))`), boolean expressions, and JSON-like nested lists. If nesting depth can reach around $10^5$ or more, the recursion may overflow the stack; convert to an explicit stack (shunting-yard) in that case.

## 5. Simulation

### 5.1 Huge step counts and cycle detection

A common regional twist: "the robot follows these rules; where is it after K = $10^{18}$ steps?" If the state space is finite (say, R·C·4 states for position and direction), the sequence of states must eventually repeat, and from the first repeat onwards it is periodic.

Record the time `first[s]` when each state `s` was first seen. When at time `t` you are in a state seen before, the cycle length is `len = t - first[s]`. The remaining `K - t` steps reduce to `(K - t) mod len` steps, which you simulate directly. Total work is O(number of states).

The crucial condition is that the state is **complete**: the next state must depend only on the current state. If the robot also carries a counter mod 3, or the grid changes as it moves, those belong in the state too. An incomplete state gives a false period and a wrong answer. If the state is large or structured, store it in a `map<State, long long>` or hash it.

Alternative: Floyd's tortoise-and-hare uses O(1) memory but is rarely needed in contests.

### 5.2 Event-driven simulation

Bank queues, elevators, network packets and similar problems are best simulated by processing events in time order with a priority queue of tuples `(time, priority, id)`. Statements usually say what happens when events coincide ("arrivals before departures", "lower id first"); encode that rule in the tuple order so the heap enforces it automatically. Use `long long` time when durations accumulate.

## 6. Reference implementations

### 6.1 Codeforces 1B — Spreadsheets

```cpp
#include <bits/stdc++.h>
using namespace std;

string toLetters(long long c) {          // 1 -> A, 26 -> Z, 27 -> AA
    string s;
    while (c > 0) {
        c--;                             // bijective base 26: shift to 0..25
        s += char('A' + c % 26);
        c /= 26;
    }
    reverse(s.begin(), s.end());
    return s;
}

long long fromLetters(const string& s) {
    long long c = 0;
    for (char ch : s) c = c * 26 + (ch - 'A' + 1);
    return c;
}

int main() {
    int n;
    cin >> n;
    while (n--) {
```

```
        string s;
        cin >> s;
        size_t cpos = s.find('C', 1);
        bool rc = s[0] == 'R' && isdigit(s[1]) && cpos != string::npos;
        if (rc) {
            long long row = stoll(s.substr(1, cpos - 1));
            long long col = stoll(s.substr(cpos + 1));
            cout << toLetters(col) << row << '\n';
        } else {
            size_t i = 0;
            while (isalpha(s[i])) i++;
            cout << 'R' << s.substr(i) << 'C' << fromLetters(s.substr(0, i)) << '\n';
        }
    }
}
```

On input `R23C55`, `BC23`, `R1C26`, `AZ1` it prints `BC23`, `R23C55`, `Z1`, `R1C52`.

## 6.2 Expression evaluator (recursive descent)

Reads one expression per line and prints its value (integer division truncates toward zero).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Parser {
    string s;
    size_t pos = 0;
    char peek() {                          // skip spaces, look at next char
        while (pos < s.size() && s[pos] == ' ') pos++;
        return pos < s.size() ? s[pos] : '\0';
    }
    ll expr() {
        ll v = term();
        while (peek() == '+' || peek() == '-') {
            char op = s[pos++];
            ll r = term();
            v = (op == '+') ? v + r : v - r;
        }
        return v;
    }
    ll term() {
        ll v = factor();
        while (peek() == '*' || peek() == '/') {
            char op = s[pos++];
            ll r = factor();
            v = (op == '*') ? v * r : v / r;
        }
        return v;
    }
    ll factor() {
        char c = peek();
```

```
        if (c == '-') { pos++; return -factor(); }
        if (c == '(') { pos++; ll v = expr(); peek(); pos++; return v; }  // skip ')'
        ll v = 0;
        while (pos < s.size() && isdigit(s[pos])) v = v * 10 + (s[pos++] - '0');
        return v;
    }
};

int main() {
    string line;
    while (getline(cin, line)) {
        if (line.empty()) continue;
        Parser p{line};
        cout << p.expr() << '\n';
    }
}
```

Input lines `1 + 2 * 3`, `(1+2)*3`, `-(4 - 10) / 4`, `2*-3 - -4`, `100/7/2` give `7`, `9`, `1`, `-2`, `7`.

### 6.3 One 2048 move (Kattis `2048`)

Input: four lines of four tile values (0 for empty), then a direction 0 = left, 1 = up, 2 = right, 3 = down.

```
#include <bits/stdc++.h>
using namespace std;
using Grid = array<array<int, 4>, 4>;

Grid rotateCCW(const Grid& g) {          // new[i][j] = old[j][3-i]
    Grid r;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++) r[i][j] = g[j][3 - i];
    return r;
}

void slideLeft(array<int, 4>& row) {
    vector<int> v, out;
    for (int x : row) if (x) v.push_back(x);                // compress
    for (size_t i = 0; i < v.size(); i++) {                 // merge once
        if (i + 1 < v.size() && v[i] == v[i + 1]) { out.push_back(2 * v[i]); i++; }
        else out.push_back(v[i]);
    }
    for (int i = 0; i < 4; i++) row[i] = i < (int)out.size() ? out[i] : 0;  // pad
}

int main() {
    Grid g;
    for (auto& row : g) for (int& x : row) cin >> x;
    int dir;
    cin >> dir;
    for (int k = 0; k < dir; k++) g = rotateCCW(g);
    for (auto& row : g) slideLeft(row);
```

```
    for (int k = dir; k % 4 != 0; k++) g = rotateCCW(g);
    for (auto& row : g)
        for (int j = 0; j < 4; j++) cout << row[j] << (j == 3 ? '\n' : ' ');
}
```

### 6.4 Robot after K steps with cycle detection

A robot starts at `S` facing north. Each step it moves forward if the cell ahead is inside the grid and not `#`, otherwise it turns right. Print its cell (1-based) after K ≤ $10^{18}$ steps. This version was stress-tested against the plain O(K) loop on random small grids.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int R, C;
    long long K;
    cin >> R >> C >> K;
    vector<string> g(R);
    for (auto& row : g) cin >> row;
    int r = 0, c = 0, d = 0;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (g[i][j] == 'S') r = i, c = j;
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    auto step = [&]() {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr >= 0 && nr < R && nc >= 0 && nc < C && g[nr][nc] != '#') r = nr, c = nc;
        else d = (d + 1) % 4;
    };
    vector<long long> seen(4LL * R * C, -1);
    for (long long t = 0; t < K; t++) {
        int s = (r * C + c) * 4 + d;
        if (seen[s] != -1) {
            long long rem = (K - t) % (t - seen[s]);
            while (rem--) step();
            break;
        }
        seen[s] = t;
        step();
    }
    cout << r + 1 << ' ' << c + 1 << '\n';
}
```

## 7. Common mistakes

- **Leftover newline** when mixing `cin >>` with `getline`.
- **1-based vs 0-based** indices: convert once at input time and once at output time, nowhere else.

- **Incomplete state** in cycle detection, giving a wrong period.
- **`int` step counters** when K can be $10^{18}$.
- **Ignoring tie rules** for simultaneous events.
- **Copy-pasted direction code** with one copy wrong.
- **Right-recursive grammar** for left-associative operators (`8 - 3 - 2` evaluated as 7).
- **Output format**: `-0.00`, missing blank lines between test cases, wrong capitalisation.
- **Deep recursion** in parsers or flood fills on $10^6$ cells: use an explicit stack or BFS.

## 8. Practice problems

- Codeforces 1B — Spreadsheets: https://codeforces.com/problemset/problem/1/B
- Codeforces 4C — Registration System: https://codeforces.com/problemset/problem/4/C
- Kattis `2048`: https://open.kattis.com/problems/2048
- Kattis `robotturtles` — Robot Turtles (parsing plus BFS on states): https://open.kattis.com/problems/robotturtles
- UVa 727 — Equation (infix to postfix)
- UVa 10267 — Graphical Editor (command interpreter, flood fill)
- UVa 10196 — Check The Check (chess move generation with direction arrays)
- UVa 10142 — Australian Voting (multi-round rules, careful parsing)
- CSES 1192 — Counting Rooms (grid traversal without recursion): https://cses.fi/problemset/task/1192

The UVa problems are on the UVa Online Judge (https://onlinejudge.org); 10267, 10196 and 10142 also appear in the *Programming Challenges* book.

## 9. Further reading

- Steven Halim et al., *Competitive Programming 4*, Section 1.4 and Chapter 6 (ad hoc, string processing, simulation problems).
- Robert Nystrom, *Crafting Interpreters*, chapter "Parsing Expressions" (free online) — a clear treatment of recursive descent and precedence.
- Year Curriculum Week 35 (debugging, stress testing) — apply it to every clever simulation shortcut.